Practical 8: Decision Trees and its Maxdepth

Problem statement - Lets have fun with building Decision Trees: You are building the decision tree classifier with the Scikit learn. Download the data of Pima Indian diabetes from Kaggle. Feature Selection - you need to divide given columns into two types of variables dependent(or target variable) and independent variable(or feature variables). Split the data and then build a decision tree. Use visualisation to showcase the pictorial representation. Find the Entropy, Information gain and Gini Index to support why you choose the specific root node.

Solution - Here is the complete code.

Conclusion - The data was divided into independent features and the dependent target variable, followed by an 80:20 train-test split. Entropy, Information Gain, and Gini Index were used to evaluate the quality of possible splits and justify the selection of the root node. Information Gain identifies the split that provides the greatest reduction in uncertainty, while Gini Index measures the impurity of the resulting nodes. The Decision Tree visualization demonstrates the classification process, and model performance was evaluated using accuracy, confusion matrix, precision, recall, and F1-score. Thus, the experiment demonstrates how statistical measures of impurity can support and explain the construction of a Decision Tree classification model.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

from sklearn.feature_selection import mutual_info_classif

import warnings
warnings.filterwarnings("ignore")
```

```
url = "https://raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv"

df = pd.read_csv(url)

df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Next steps: Generate code with df | New interactive sheet

```
print("Shape:", df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nFirst 5 rows:")
display(df.head())
```

```
Shape: (768, 9)

Columns:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']

First 5 rows:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
```

```
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	76
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	

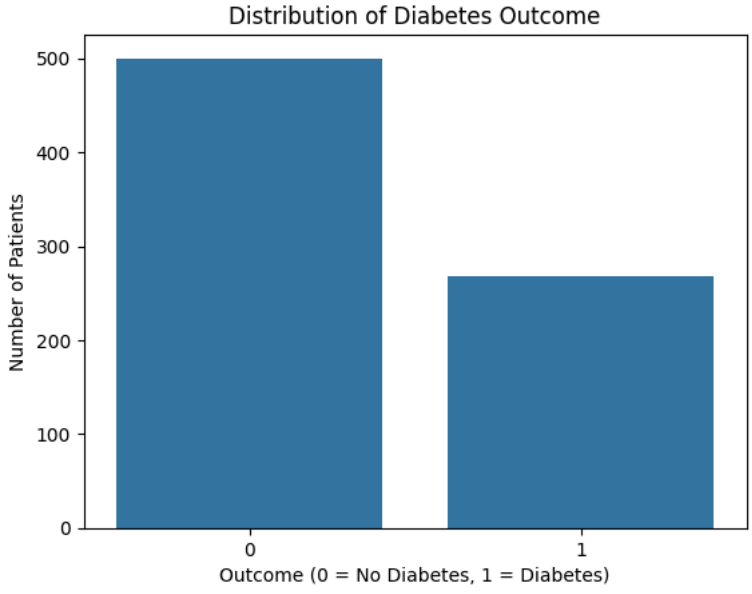
```
print(df["Outcome"].value_counts())

sns.countplot(x="Outcome", data=df)

plt.title("Distribution of Diabetes Outcome")
plt.xlabel("Outcome (0 = No Diabetes, 1 = Diabetes)")
plt.ylabel("Number of Patients")

plt.show()
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
```




```
X = df.drop("Outcome", axis=1)

y = df["Outcome"]

print("Independent variables (X):")
print(X.columns.tolist())

print("\nDependent variable (y):")
print("Outcome")
```

```
Independent variables (X):
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

Dependent variable (y):
Outcome
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.20, random_state=42, stratify=y)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (614, 8)
Testing data: (154, 8)
```

```
dt_entropy = DecisionTreeClassifier(criterion="entropy", random_state=42)
dt_entropy.fit(X_train, y_train)
```

```
DecisionTreeClassifier(criterion='entropy', random_state=42)
```

```
y_pred = dt_entropy.predict(X_test)

print("Predictions:")
print(y_pred[:20])
```

```
Predictions:
[1 0 0 1 0 0 0 1 0 1 1 1 0 0 0 0 0 0 1 1]
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Decision Tree Accuracy:", accuracy)
print("Accuracy percentage:", accuracy * 100, "%")
```

```
Decision Tree Accuracy: 0.7012987012987013
Accuracy percentage: 70.12987012987013 %
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.75      0.80      0.78       100
           1       0.58      0.52      0.55        54

    accuracy                           0.70       154
   macro avg       0.67      0.66      0.66       154
weighted avg       0.69      0.70      0.70       154
```

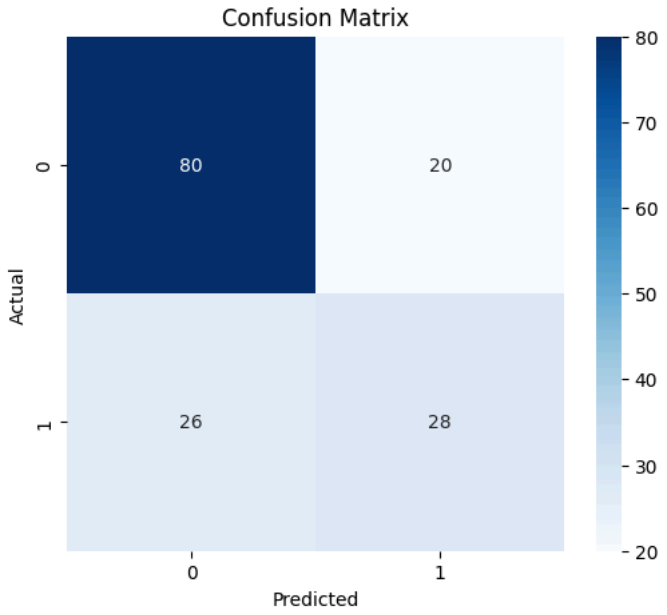
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

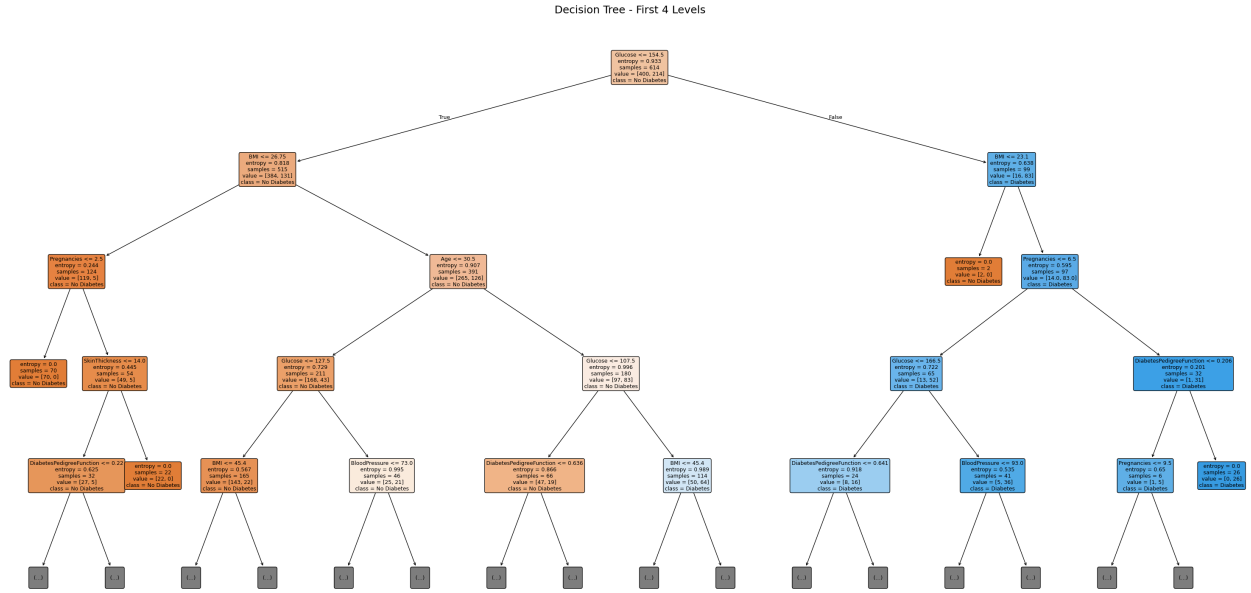


```
plt.figure(figsize=(30, 15))

plot_tree(
    dt_entropy,
    feature_names=X.columns,
    class_names=["No Diabetes", "Diabetes"],
    filled=True,
    rounded=True,
    max_depth=4,
    fontsize=9
)

plt.title("Decision Tree - First 4 Levels", fontsize=18)

plt.tight_layout()
plt.show()
```




```
# Find the root node of the Decision Tree

root_index = dt_entropy.tree_.feature[0]
root_feature = X.columns[root_index]
root_threshold = dt_entropy.tree_.threshold[0]

print("Root Node Feature:", root_feature)
print("Root Node Threshold:", root_threshold)
```

```
Root Node Feature: Glucose
Root Node Threshold: 154.5
```

```
# ENTROPY, INFORMATION GAIN AND GINI INDEx

import numpy as np
import pandas as pd

# Entropy function
def entropy(y):
    probabilities = y.value_counts(normalize=True)
    return -np.sum(probabilities * np.log2(probabilities))


# Gini Index function
def gini(y):
    probabilities = y.value_counts(normalize=True)
    return 1 - np.sum(probabilities ** 2)
```

```
# Use training data
train_data = X_train.copy()
train_data["Outcome"] = y_train

# Parent entropy and Gini
parent_entropy = entropy(train_data["Outcome"])
parent_gini = gini(train_data["Outcome"])

results = []

# Find best threshold for every feature
for feature in X_train.columns:

    values = np.sort(train_data[feature].unique())

    best_ig = -np.inf
    best_threshold = None
    best_gini = None
    best_entropy = None

    for i in range(len(values) - 1):

        threshold = (values[i] + values[i + 1]) / 2

        left = train_data[
            train_data[feature] <= threshold
        ]["Outcome"]

        right = train_data[
            train_data[feature] > threshold
        ]["Outcome"]

        if len(left) == 0 or len(right) == 0:
            continue

        # Weighted entropy after split
        weighted_entropy = (
            (len(left) / len(train_data)) * entropy(left)
            +
            (len(right) / len(train_data)) * entropy(right)
        )

        # Information Gain
        information_gain = parent_entropy - weighted_entropy

        # Weighted Gini
        weighted_gini = (
            (len(left) / len(train_data)) * gini(left)
            +
            (len(right) / len(train_data)) * gini(right)
        )

        # Select maximum Information Gain
        if information_gain > best_ig:

            best_ig = information_gain
            best_threshold = threshold
            best_gini = weighted_gini
            best_entropy = weighted_entropy

    results.append({
        "Feature": feature,
        "Best Threshold": best_threshold,
        "Weighted Entropy": best_entropy,
        "Information Gain": best_ig,
        "Weighted Gini Index": best_gini
    })


# Create table
results_df = pd.DataFrame(results)

# Sort by Information Gain
results_df = results_df.sort_values(
    "Information Gain",
    ascending=False
).reset_index(drop=True)


print("Parent Entropy:", round(parent_entropy, 4))
print("Parent Gini Index:", round(parent_gini, 4))
```

```
print("\nFeature-wise Entropy, Information Gain and Gini Index:")

display(results_df.round(4))
```

```
Parent Entropy: 0.9328
Parent Gini Index: 0.4541

Feature-wise Entropy, Information Gain and Gini Index:
```

	Feature	Best Threshold	Weighted Entropy	Information Gain	Weighted Gini Index
0	Glucose	154.5000	0.7891	0.1436	0.3619
1	BMI	29.6500	0.8587	0.0741	0.4109
2	Age	27.5000	0.8673	0.0654	0.4147
3	Pregnancies	6.5000	0.8940	0.0388	0.4287
4	Insulin	128.5000	0.8960	0.0368	0.4302
5	DiabetesPedigreeFunction	0.5275	0.9084	0.0243	0.4386
6	SkinThickness	32.5000	0.9099	0.0229	0.4393
7	BloodPressure	69.0000	0.9142	0.0185	0.4427

```
# Identify the feature with maximum Information Gain

best_feature = results_df.iloc[0]

print("BEST FEATURE ACCORDING TO INFORMATION GAIN")

print("Feature:", best_feature["Feature"])
print("Threshold:", round(best_feature["Best Threshold"], 4))
print("Information Gain:", round(best_feature["Information Gain"], 4))
print("Weighted Entropy:", round(best_feature["Weighted Entropy"], 4))
print("Weighted Gini Index:", round(best_feature["Weighted Gini Index"], 4))
```

```
BEST FEATURE ACCORDING TO INFORMATION GAIN
Feature: Glucose
Threshold: 154.5
Information Gain: 0.1436
Weighted Entropy: 0.7891
Weighted Gini Index: 0.3619
```

Interpretation - Higher Information Gain = better split, Lower Gini Index = better/purer split.

```
# Compare calculated best feature with the actual Decision Tree root

print("ACTUAL DECISION TREE ROOT")

print("Feature:", root_feature)
print("Threshold:", round(root_threshold, 4))

print("\nCALCULATED BEST FEATURE")

print("Feature:", best_feature["Feature"])
print("Threshold:", round(best_feature["Best Threshold"], 4))

if root_feature == best_feature["Feature"]:
    print("\n✓ Both methods selected the same root feature.")
else:
    print("\nThe selected features are different. Check the calculation.")
```

```
ACTUAL DECISION TREE ROOT
Feature: Glucose
Threshold: 154.5

CALCULATED BEST FEATURE
Feature: Glucose
Threshold: 154.5

✓ Both methods selected the same root feature.
```

```
# Predictions on test data

y_pred = dt_entropy.predict(X_test)

print("First 20 actual values:")
print(y_test.values[:20])

print("\nFirst 20 predicted values:")
print(y_pred[:20])
```

```
First 20 actual values:
[0 0 0 1 0 0 1 1 0 0 0 1 0 0 1 1 0 0 0 0]

First 20 predicted values:
[1 0 0 1 0 0 0 1 0 1 1 1 0 0 0 0 0 0 1 1]
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Decision Tree Accuracy:")
print(round(accuracy, 4))

print("\nAccuracy Percentage:")
print(round(accuracy * 100, 2), "%")
```

```
Decision Tree Accuracy:
0.7013

Accuracy Percentage:
70.13 %
```

Start coding or generate with AI.